Research-Paper-Intelligence-System

```
Research-Paper-Intelligence-System/
│
├── documents/
│   ├── paper1.pdf
│   ├── paper2.pdf
│   ├── paper3.pdf
│   └── paper4.pdf
│
├── vector_store/
│   ├── index.faiss
│   └── metadata.pkl
│
├── src/
│   ├── pdf_loader.py
│   ├── chunking.py
│   ├── embeddings.py
│   ├── vector_store.py
│   ├── retriever.py
│   ├── llm.py
│   └── rag_pipeline.py
│
├── evaluation/
```

```
│   └── evaluation.py
│
├── main.py
│
├── requirements.txt
│
├── .env
│
├── .env.example
│
├── .gitignore
│
└── README.md
```

Steps to implement:

Research Paper Intelligence System — Development Steps

Follow these steps **in this exact order**.

STEP 1 — Create the project

`Research-Paper-Intelligence-System/`

Create the folders:

```
documents/
vector_store/
src/
evaluation/
```

STEP 2 — Set up Python environment

In VS Code terminal:

```
python -m venv venv
```

Activate:

```
.\venv\Scripts\Activate.ps1
```

Install:

```
pip install pymupdf sentence-transformers faiss-cpu google-genai
python-dotenv numpy
```

Save dependencies:

```
pip freeze > requirements.txt
```

STEP 3 — Add research papers

Put your PDFs inside:(search in google for few researcch papers)

```
documents/
├── paper1.pdf
├── paper2.pdf
├── paper3.pdf
└── paper4.pdf
```

Start with **2–4 papers**.

Where to get the PDFs

You can download research papers from:

- Google Scholar
- arXiv
- IEEE Xplore
- Semantic Scholar

STEP 4 — PDF extraction

Create:

`src/pdf_loader.py`

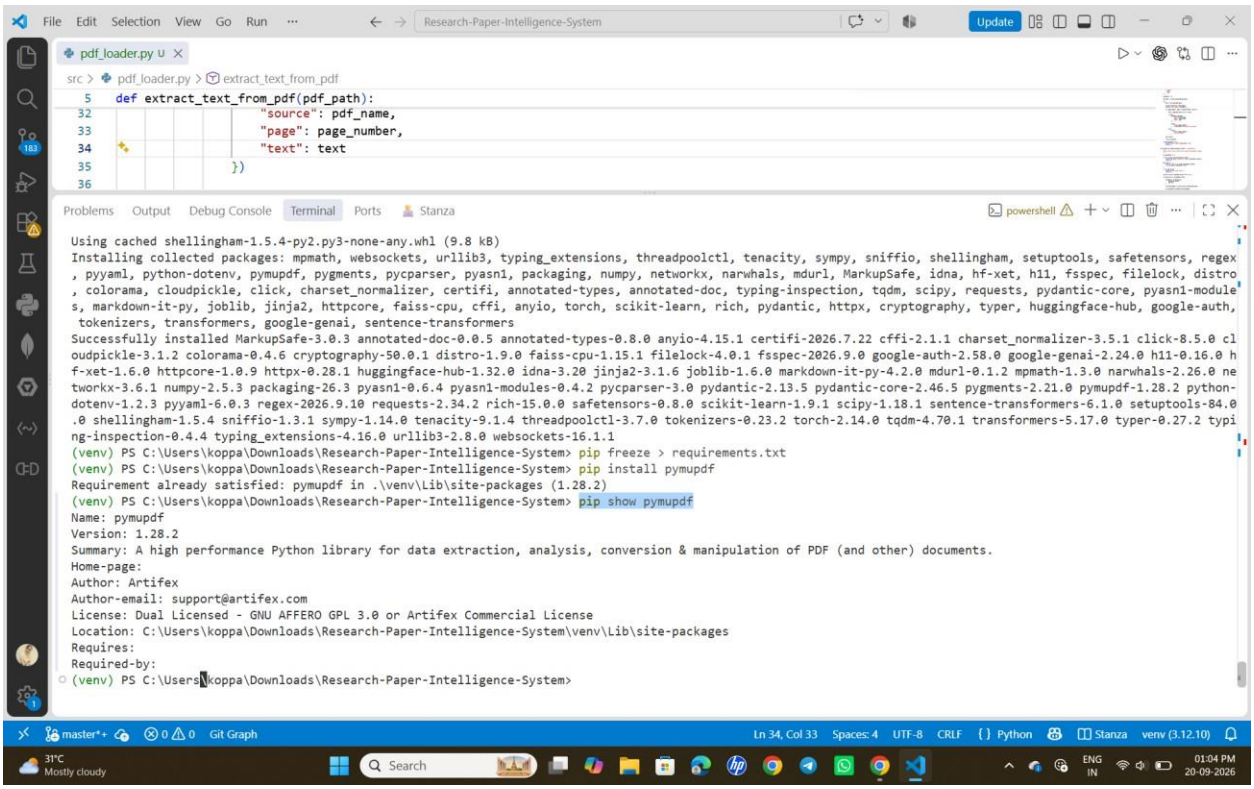
Purpose:

```
PDF
 ↓
PyMuPDF
 ↓
Text + Page Number
```

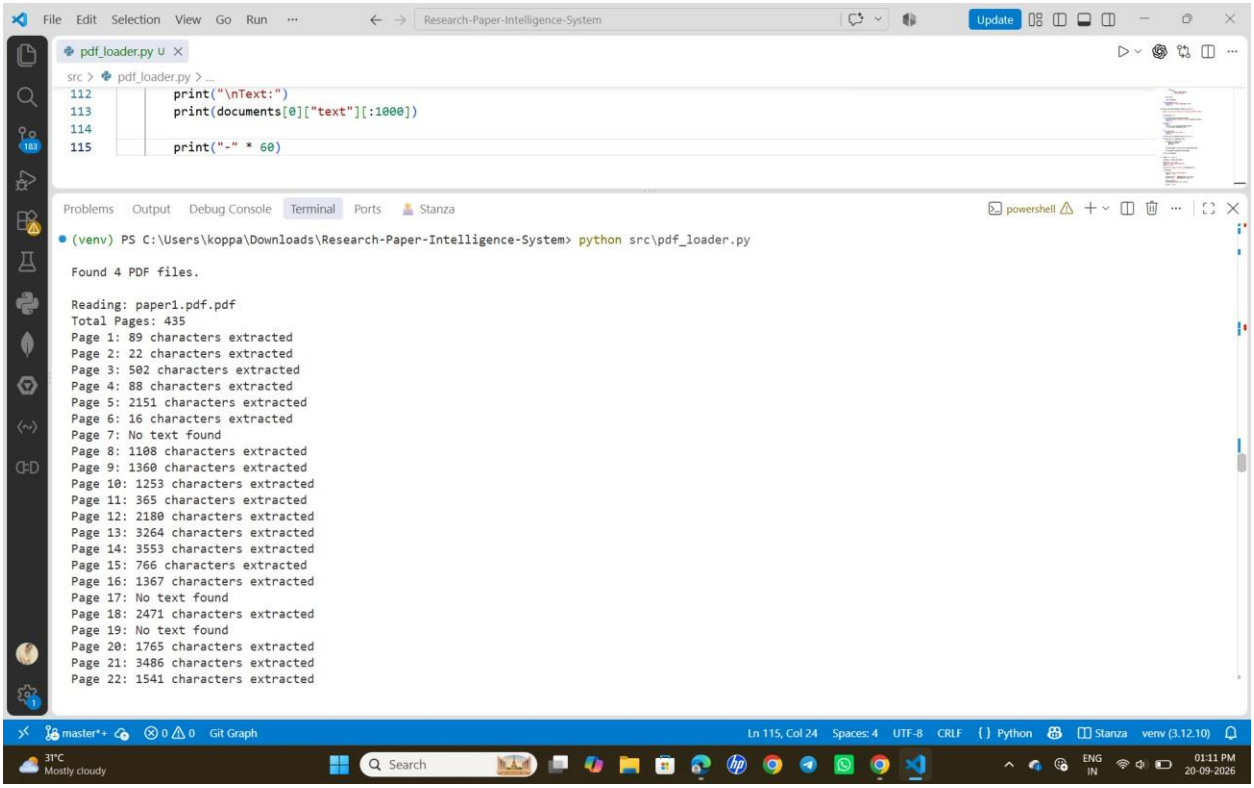
Output should preserve:

```
paper1.pdf
Page 1 → text
Page 2 → text
Page 3 → text
...
```

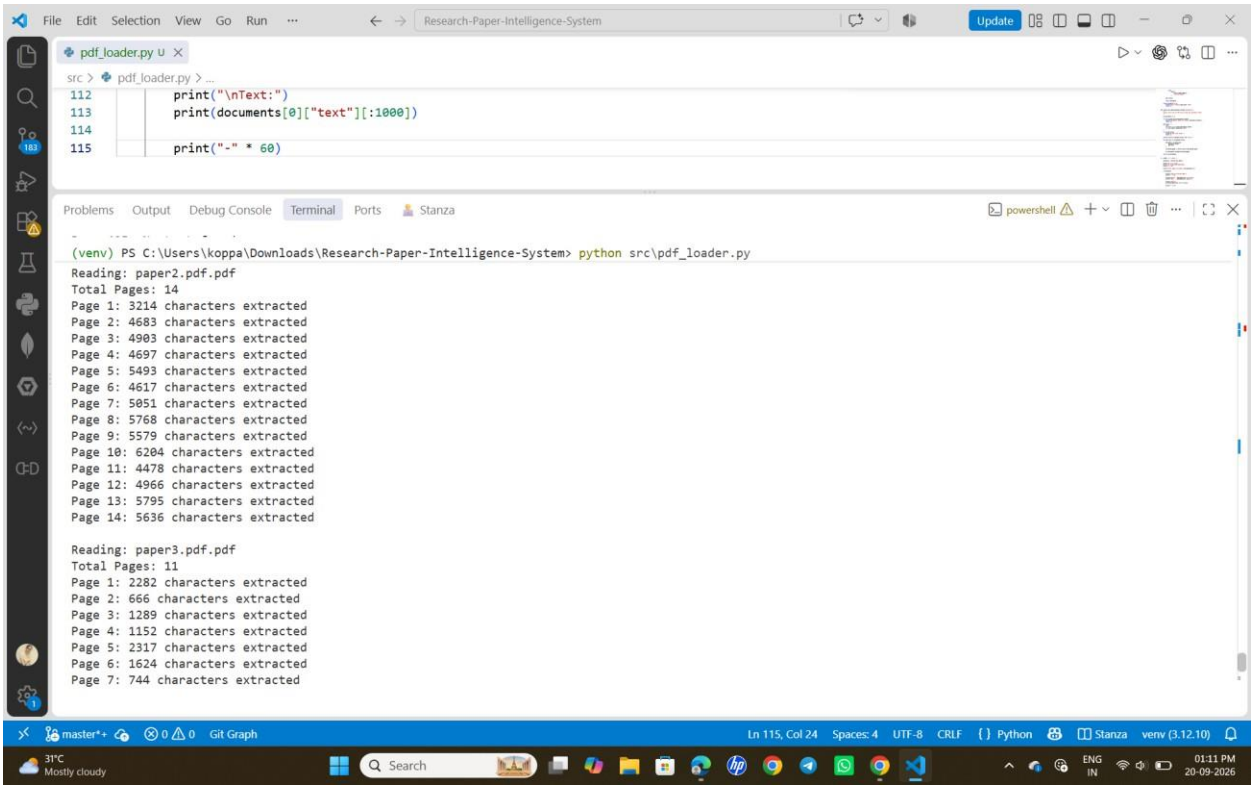
Test this first.



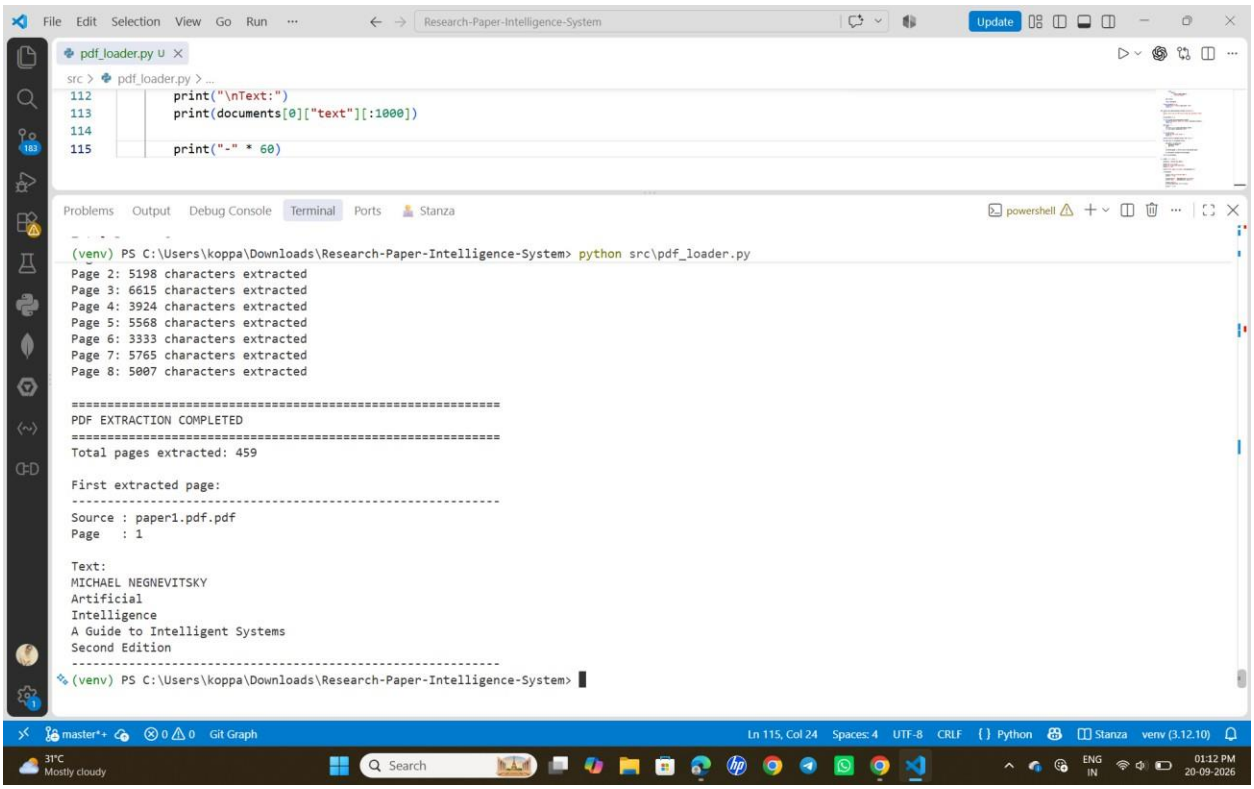
Output 1(page1)



Output 2 & 3(page2.pdf & paper3.pdf)



Output 4(paper4.pdf)



STEP 5 — Chunking

Create:

`src/chunking.py`

Convert extracted text into smaller chunks.

```
Full Paper
    ↓
Chunk 1
Chunk 2
Chunk 3
Chunk 4
...
```

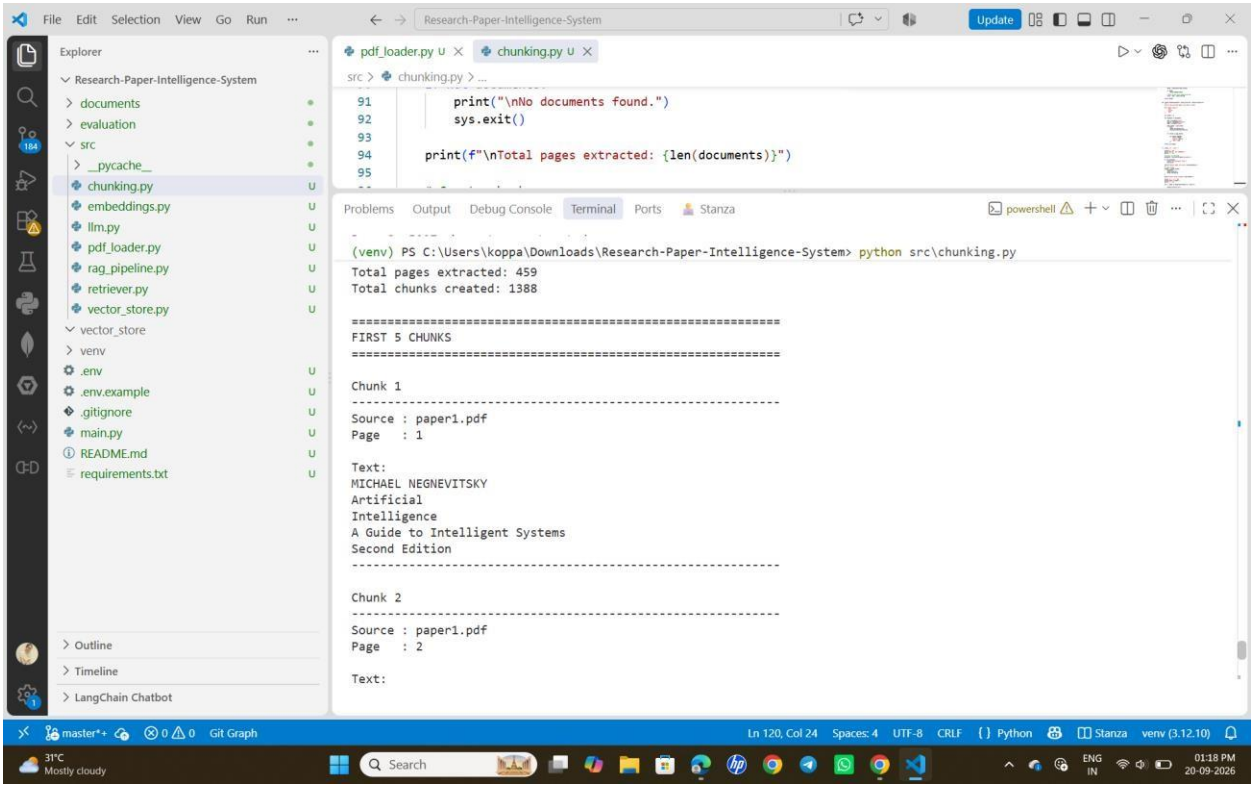
Each chunk should contain:

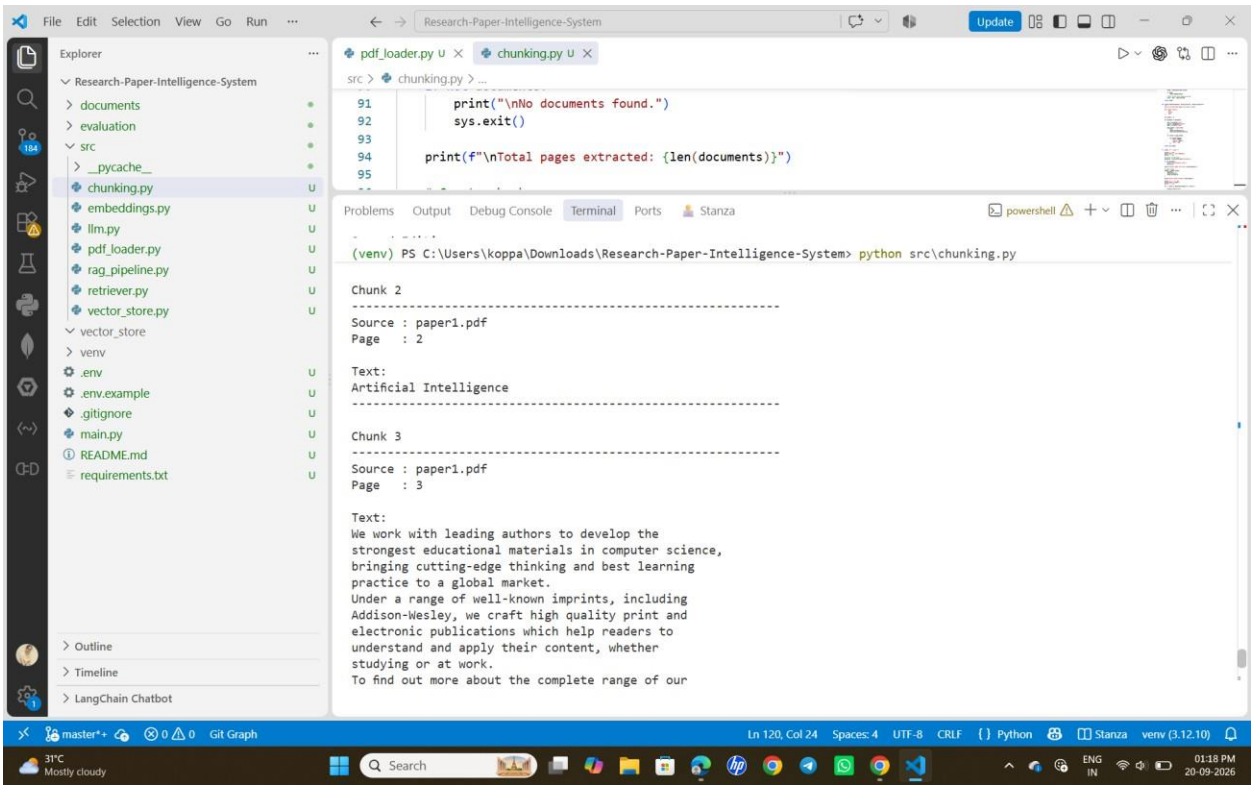
text
source
page

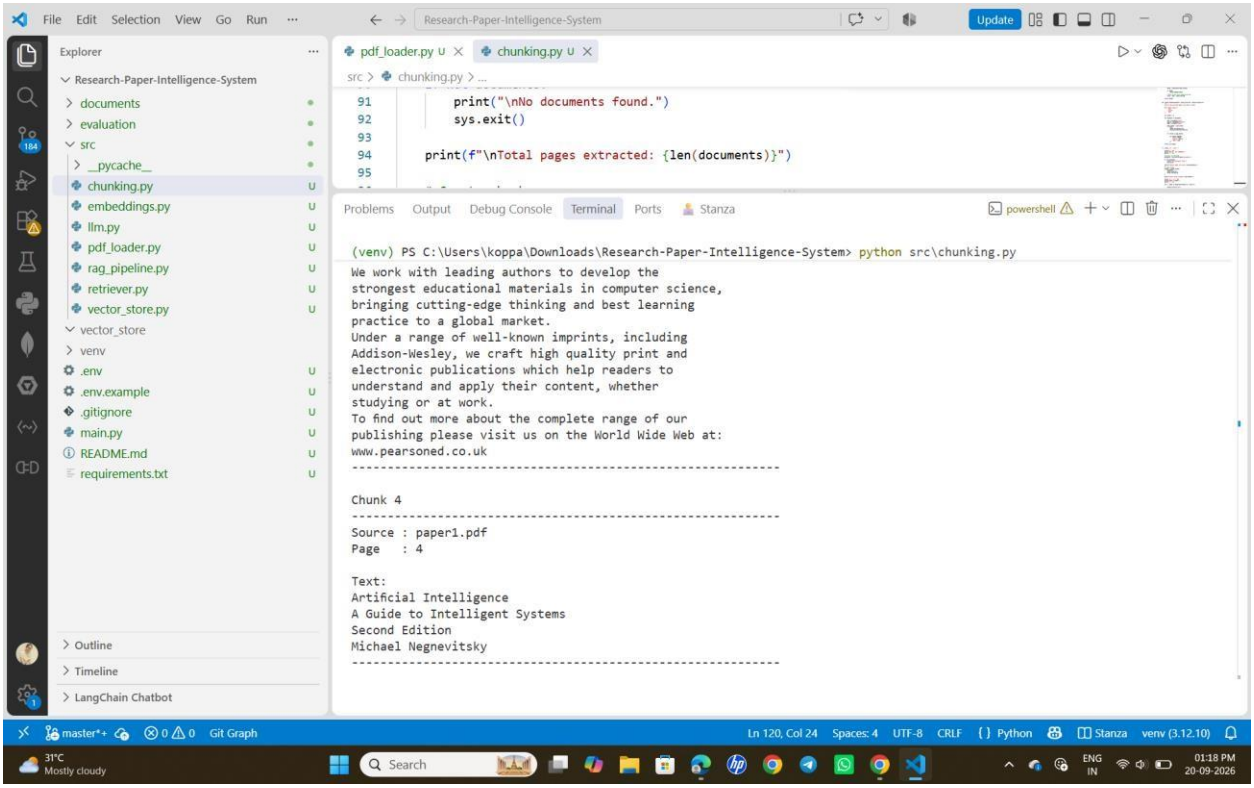
Example:

```
{
    "text": "...",
    "source": "paper1.pdf",
    "page": 5
}
```

Output:




File Edit Selection View Go Run
Research-Paper-Intelligence-System
Update
Explorer
Research-Paper-Intelligence-System
documents
evaluation
src
__pycache__
chunking.py
embeddings.py
llm.py
pdf_loader.py
rag_pipeline.py
retriever.py
vector_store.py
vector_store
venv
.env
.env.example
.gitignore
main.py
README.md
requirements.txt
pdf_loader.py U
chunking.py U
src > chunking.py > ...
91 print("\nNo documents found.")
92 sys.exit()
94 print(f"\nTotal pages extracted: {len(documents)}")
Problems Output Debug Console Terminal Ports Stanza
powershell
(venv) PS C:\Users\koppa\Downloads\Research-Paper-Intelligence-System> python src\chunking.py
Chunk 2
Source : paper1.pdf
Page : 2
Text:
Artificial Intelligence
Chunk 3
Source : paper1.pdf
Page : 3
Text:
We work with leading authors to develop the
strongest educational materials in computer science,
bringing cutting-edge thinking and best learning
practice to a global market.
Under a range of well-known imprints, including
Addison-Wesley, we craft high quality print and
electronic publications which help readers to
understand and apply their content, whether
studying or at work.
To find out more about the complete range of our
Outline
Timeline
LangChain Chatbot
master*+ 0 0 Git Graph
Ln 120, Col 24 Spaces: 4 UTF-8 CRLF Python Stanza venv (3.12.10)
31°C Mostly cloudy
Search
ENG IN
01:18 PM 20-09-2026


File Edit Selection View Go Run
Research-Paper-Intelligence-System
Update
Explorer
Research-Paper-Intelligence-System
documents
evaluation
src
__pycache__
chunking.py
embeddings.py
llm.py
pdf_loader.py
rag_pipeline.py
retriever.py
vector_store.py
vector_store
venv
.env
.env.example
.gitignore
main.py
README.md
requirements.txt
pdf_loader.py U
chunking.py U
src > chunking.py > ...
91 print("\nNo documents found.")
92 sys.exit()
94 print(f"\nTotal pages extracted: {len(documents)}")
Problems Output Debug Console Terminal Ports Stanza
powershell
(venv) PS C:\Users\koppa\Downloads\Research-Paper-Intelligence-System> python src\chunking.py
We work with leading authors to develop the
strongest educational materials in computer science,
bringing cutting-edge thinking and best learning
practice to a global market.
Under a range of well-known imprints, including
Addison-Wesley, we craft high quality print and
electronic publications which help readers to
understand and apply their content, whether
studying or at work.
To find out more about the complete range of our
publishing please visit us on the World Wide Web at:
www.pearsoned.co.uk
Chunk 4
Source : paper1.pdf
Page : 4
Text:
Artificial Intelligence
A Guide to Intelligent Systems
Second Edition
Michael Negnevitsky
Outline
Timeline
LangChain Chatbot
master*+ 0 0 Git Graph
Ln 120, Col 24 Spaces: 4 UTF-8 CRLF Python Stanza venv (3.12.10)
31°C Mostly cloudy
Search
ENG IN
01:18 PM 20-09-2026

```
(venv) PS C:\Users\koppa\Downloads\Research-Paper-Intelligence-System> python src\chunking.py
------------------------------------------------------------
Source : paper1.pdf
Page   : 5

Text:
Pearson Education Limited
Edinburgh Gate
Harlow
Essex CM20 2JE
England
and Associated Companies throughout the World.
Visit us on the World Wide Web at:
www.pearsoned.co.uk
First published 2002
Second edition published 2005
# Pearson Education Limited 2002
The right of Michael Negnevitsky to be identified as author of this Work has been asserted
by the author in accordance with the Copyright, Designs and Patents Act 1988.
All rights reserved. No part of this publication may be reproduced, stored in a retrieval
system, or transmitted in any form or by any means, electronic, mechanical,
photocopying, recording or otherwise, without either the prior written permission of the
publisher or a licence permitting restricted copying in the United Kingdom issued by the
Copyright Licensing Agency Ltd, 90 Tottenham Court Road, London W1T 4LP.
The programs in this book have been included for their instructional value. They have been
tested with care but are not guaranteed for any particular purpose.
------------------------------------------------------------
```

Structure upto step1 to step5

4 PDF Papers

↓

PyMuPDF

↓

459 Pages

↓

Text Extraction

↓

Chunking

↓

Each chunk contains:

├── text

├── source

└── page

STEP 6 — Generate embeddings

Create:

`src/embeddings.py`

Use:

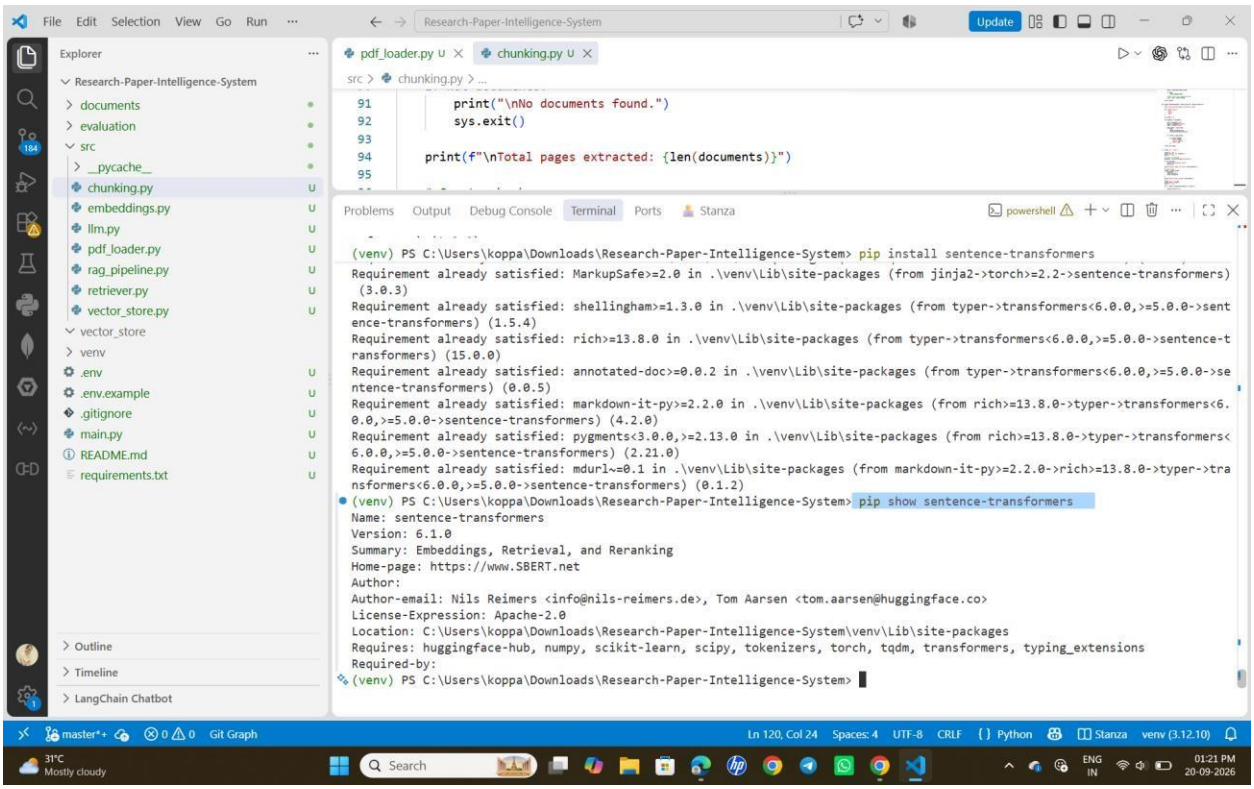
```
Sentence Transformers
all-MiniLM-L6-v2
```

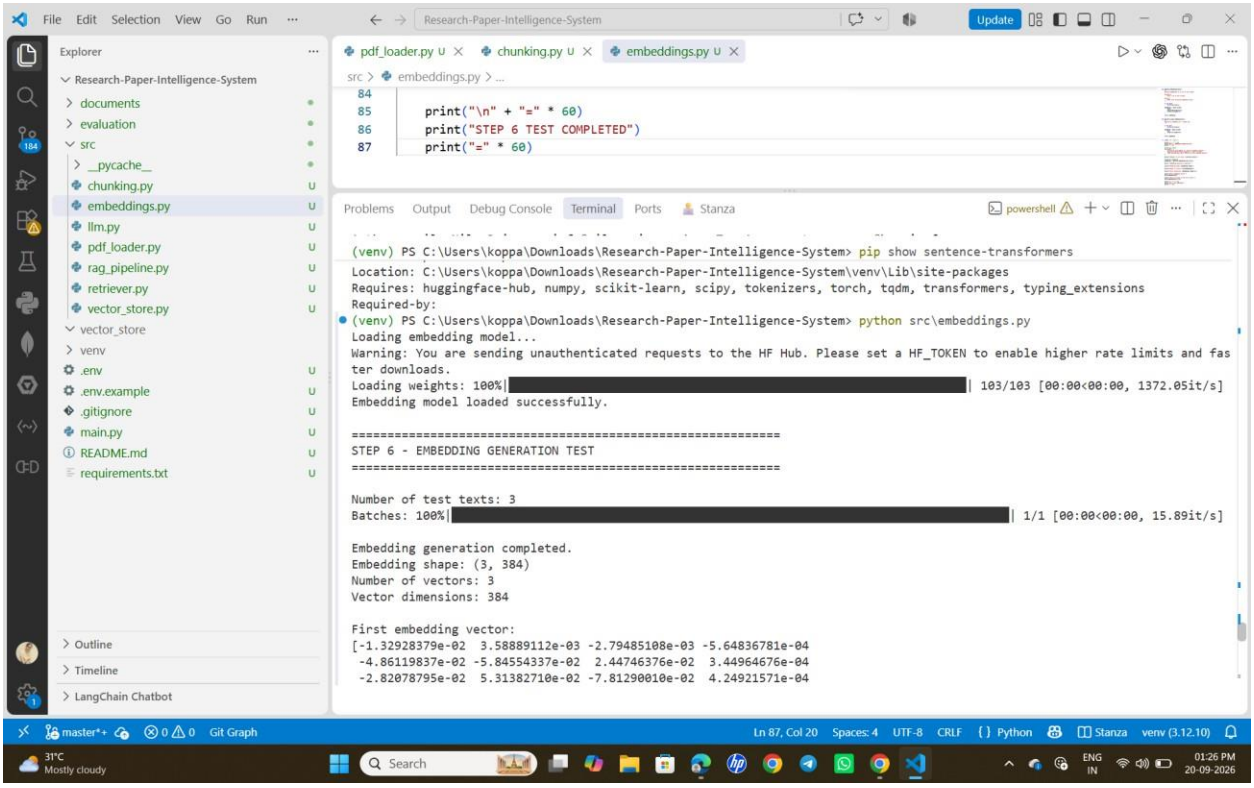
Flow:

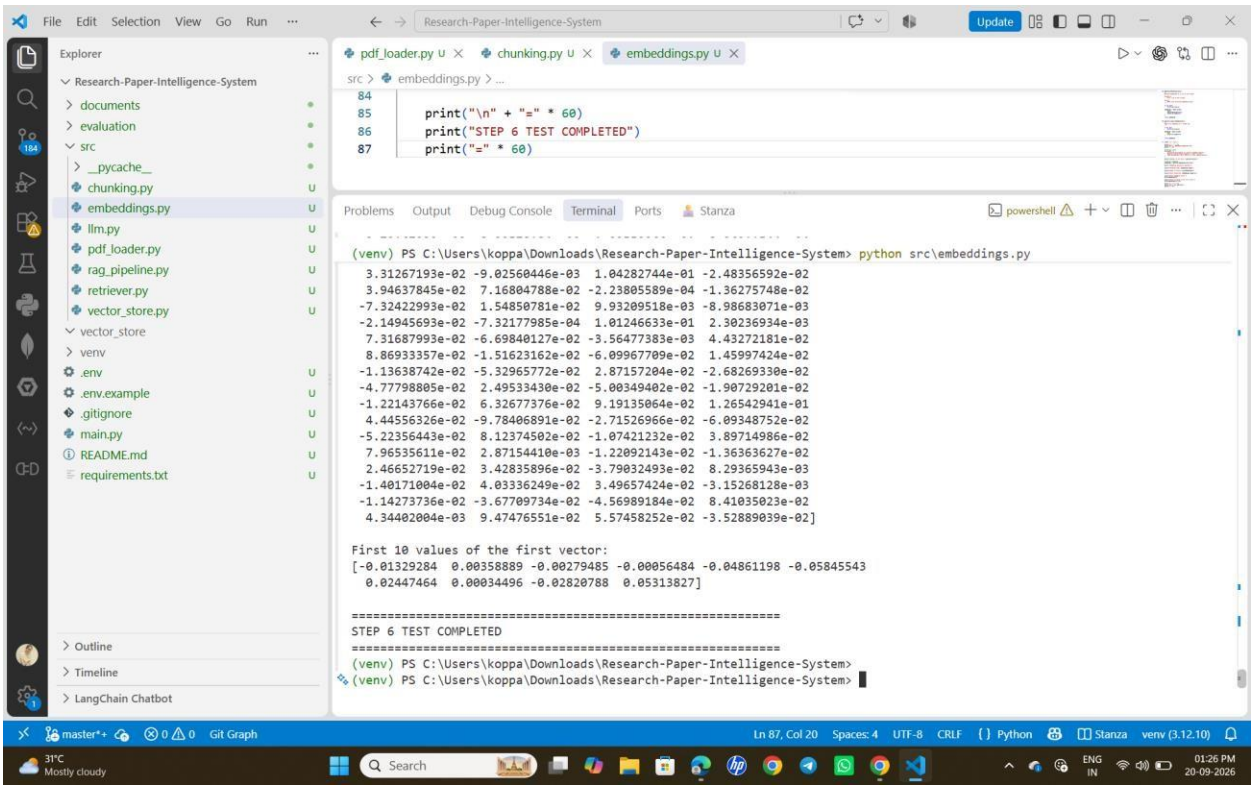
```
Text Chunk
    ↓
Sentence Transformer
    ↓
Embedding Vector
```

For `all-MiniLM-L6-v2`, embeddings are typically 384-dimensional.

Output:


File Edit Selection View Go Run
Research-Paper-Intelligence-System
Update
Explorer
Research-Paper-Intelligence-System
documents
evaluation
src
__pycache__
chunking.py
embeddings.py
llm.py
pdf_loader.py
rag_pipeline.py
retriever.py
vector_store.py
vector_store
venv
.env
.env.example
.gitignore
main.py
README.md
requirements.txt
Outline
Timeline
LangChain Chatbot
pdf_loader.py U
chunking.py U
src > chunking.py > ...
print("\nNo documents found.")
sys.exit()
print(f"\nTotal pages extracted: {len(documents)}")
Problems Output Debug Console Terminal Ports Stanza
powershell
(venv) PS C:\Users\koppa\Downloads\Research-Paper-Intelligence-System> pip install sentence-transformers
Requirement already satisfied: MarkupSafe>=2.0 in .\venv\Lib\site-packages (from jinja2->torch>=2.2->sentence-transformers) (3.0.3)
Requirement already satisfied: shellingham>=1.3.0 in .\venv\Lib\site-packages (from typer->transformers<6.0.0,>=5.0.0->sentence-transformers) (1.5.4)
Requirement already satisfied: rich>=13.8.0 in .\venv\Lib\site-packages (from typer->transformers<6.0.0,>=5.0.0->sentence-transformers) (15.0.0)
Requirement already satisfied: annotated-doc>=0.0.2 in .\venv\Lib\site-packages (from typer->transformers<6.0.0,>=5.0.0->sentence-transformers) (0.0.5)
Requirement already satisfied: markdown-it-py>=2.2.0 in .\venv\Lib\site-packages (from rich>=13.8.0->typer->transformers<6.0.0,>=5.0.0->sentence-transformers) (4.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in .\venv\Lib\site-packages (from rich>=13.8.0->typer->transformers<6.0.0,>=5.0.0->sentence-transformers) (2.21.0)
Requirement already satisfied: mdurl~=0.1 in .\venv\Lib\site-packages (from markdown-it-py>=2.2.0->rich>=13.8.0->typer->transformers<6.0.0,>=5.0.0->sentence-transformers) (0.1.2)
(venv) PS C:\Users\koppa\Downloads\Research-Paper-Intelligence-System> pip show sentence-transformers
Name: sentence-transformers
Version: 6.1.0
Summary: Embeddings, Retrieval, and Reranking
Home-page: https://www.SBERT.net
Author:
Author-email: Nils Reimers <info@nils-reimers.de>, Tom Aarsen <tom.aarsen@huggingface.co>
License-Expression: Apache-2.0
Location: C:\Users\koppa\Downloads\Research-Paper-Intelligence-System\venv\Lib\site-packages
Requires: huggingface-hub, numpy, scikit-learn, scipy, tokenizers, torch, tqdm, transformers, typing_extensions
Required-by:
(venv) PS C:\Users\koppa\Downloads\Research-Paper-Intelligence-System>
master*+ 0 0 Git Graph
Ln 120, Col 24 Spaces: 4 UTF-8 CRLF Python Stanza venv (3.12.10)
31°C Mostly cloudy
Search
ENG IN
01:21 PM 20-09-2026


File Edit Selection View Go Run
Research-Paper-Intelligence-System
Update
Explorer
Research-Paper-Intelligence-System
documents
evaluation
src
__pycache__
chunking.py
embeddings.py
llm.py
pdf_loader.py
rag_pipeline.py
retriever.py
vector_store.py
vector_store
venv
.env
.env.example
.gitignore
main.py
README.md
requirements.txt
Outline
Timeline
LangChain Chatbot
pdf_loader.py U
chunking.py U
embeddings.py U
src > embeddings.py > ...
print("\n" + "=" * 60)
print("STEP 6 TEST COMPLETED")
print("=" * 60)
Problems Output Debug Console Terminal Ports Stanza
powershell
(venv) PS C:\Users\koppa\Downloads\Research-Paper-Intelligence-System> pip show sentence-transformers
Location: C:\Users\koppa\Downloads\Research-Paper-Intelligence-System\venv\Lib\site-packages
Requires: huggingface-hub, numpy, scikit-learn, scipy, tokenizers, torch, tqdm, transformers, typing_extensions
Required-by:
(venv) PS C:\Users\koppa\Downloads\Research-Paper-Intelligence-System> python src\embeddings.py
Loading embedding model...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%| 103/103 [00:00<00:00, 1372.05it/s]
Embedding model loaded successfully.
==
STEP 6 - EMBEDDING GENERATION TEST
==
Number of test texts: 3
Batches: 100%| 1/1 [00:00<00:00, 15.89it/s]
Embedding generation completed.
Embedding shape: (3, 384)
Number of vectors: 3
Vector dimensions: 384
First embedding vector:
[-1.32928379e-02 3.58889112e-03 -2.79485108e-03 -5.64836781e-04
-4.86119837e-02 -5.84554337e-02 2.44746376e-02 3.44964676e-04
-2.82078795e-02 5.31382710e-02 -7.81290010e-02 4.24921571e-04
master*+ 0 0 Git Graph
Ln 87, Col 20 Spaces: 4 UTF-8 CRLF Python Stanza venv (3.12.10)
31°C Mostly cloudy
Search
ENG IN
01:26 PM 20-09-2026



Structure:

Text Chunk

↓

all-MiniLM-L6-v2

↓

[0.012, -0.045, 0.078, ...]

↓

384 dimensions

STEP 7 — Create FAISS vector store

Create:

`src/vector_store.py`

Store the embeddings in FAISS.

The result should be:

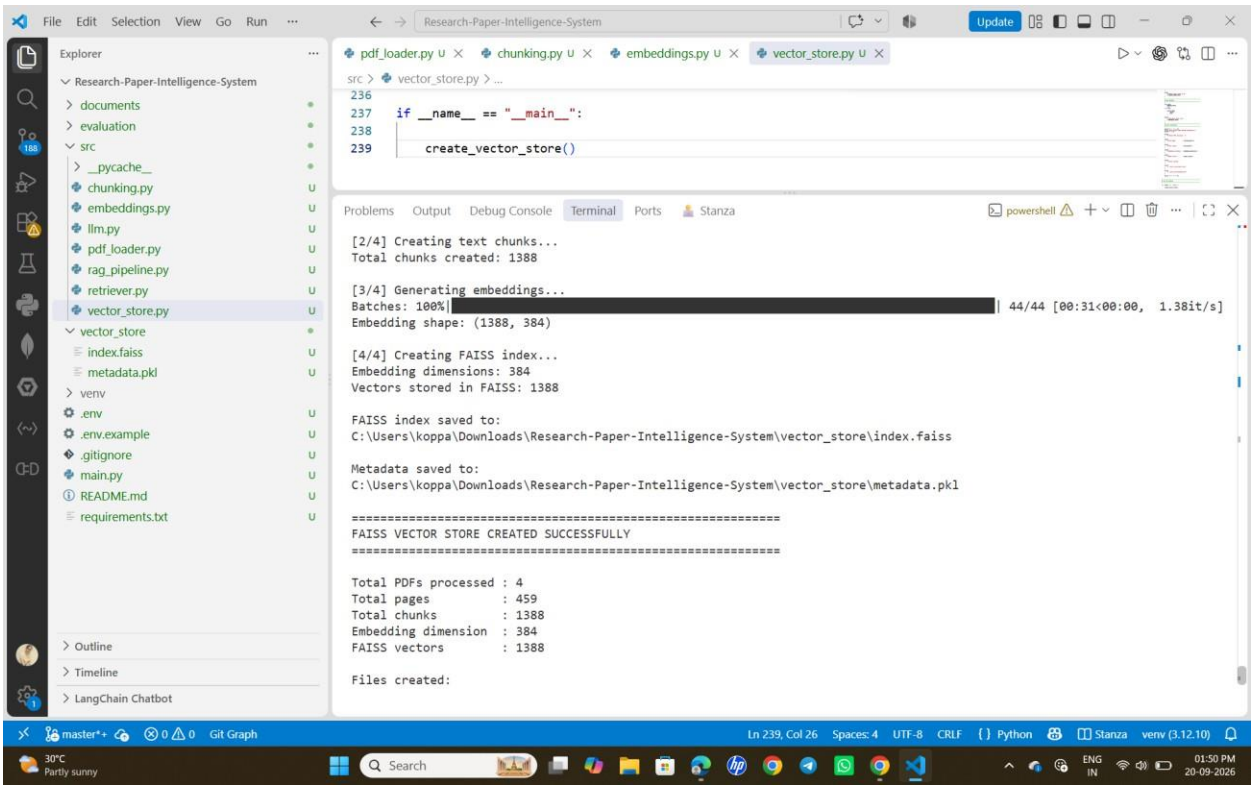
```
vector_store/
├── index.faiss
└── metadata.pkl
```

`index.faiss` → stores/searches vectors.

`metadata.pkl` → stores:

```
chunk  text
paper  name
page number
```

```
Output:
```



STEP 8 — Create the retriever

Create:

`src/retriever.py`

User asks:

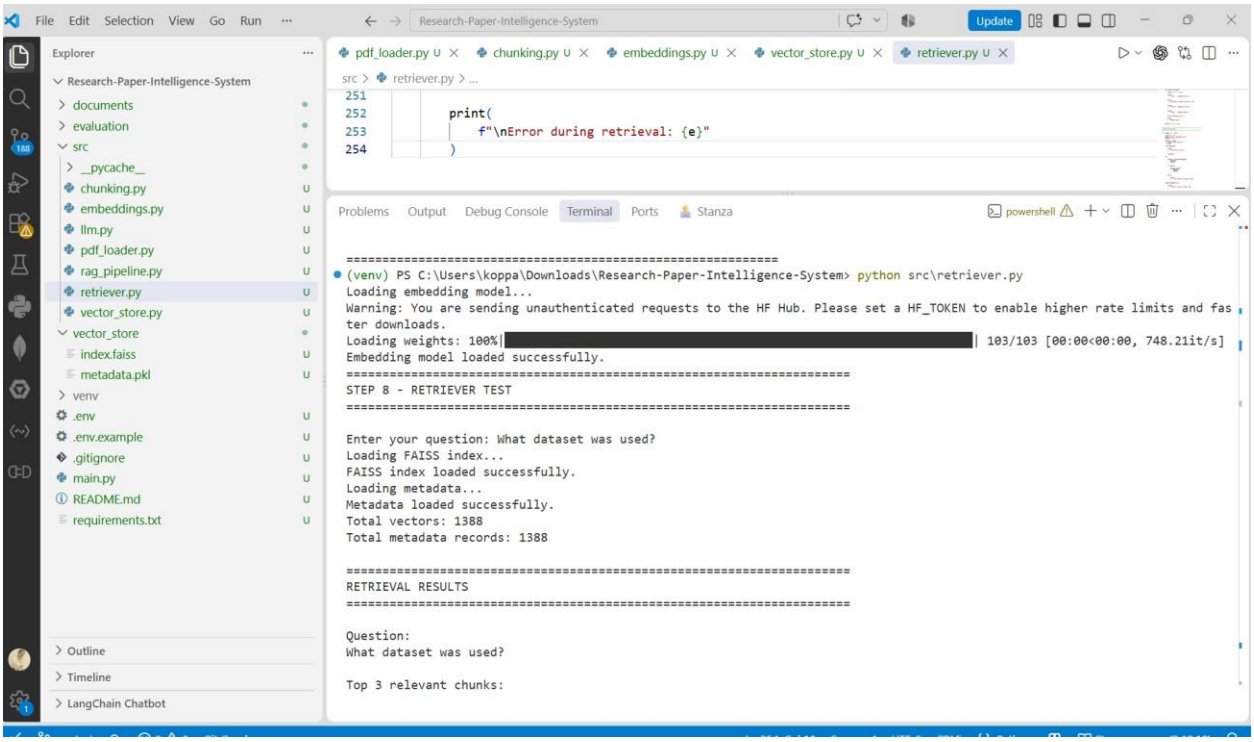
```
What dataset was used?
```

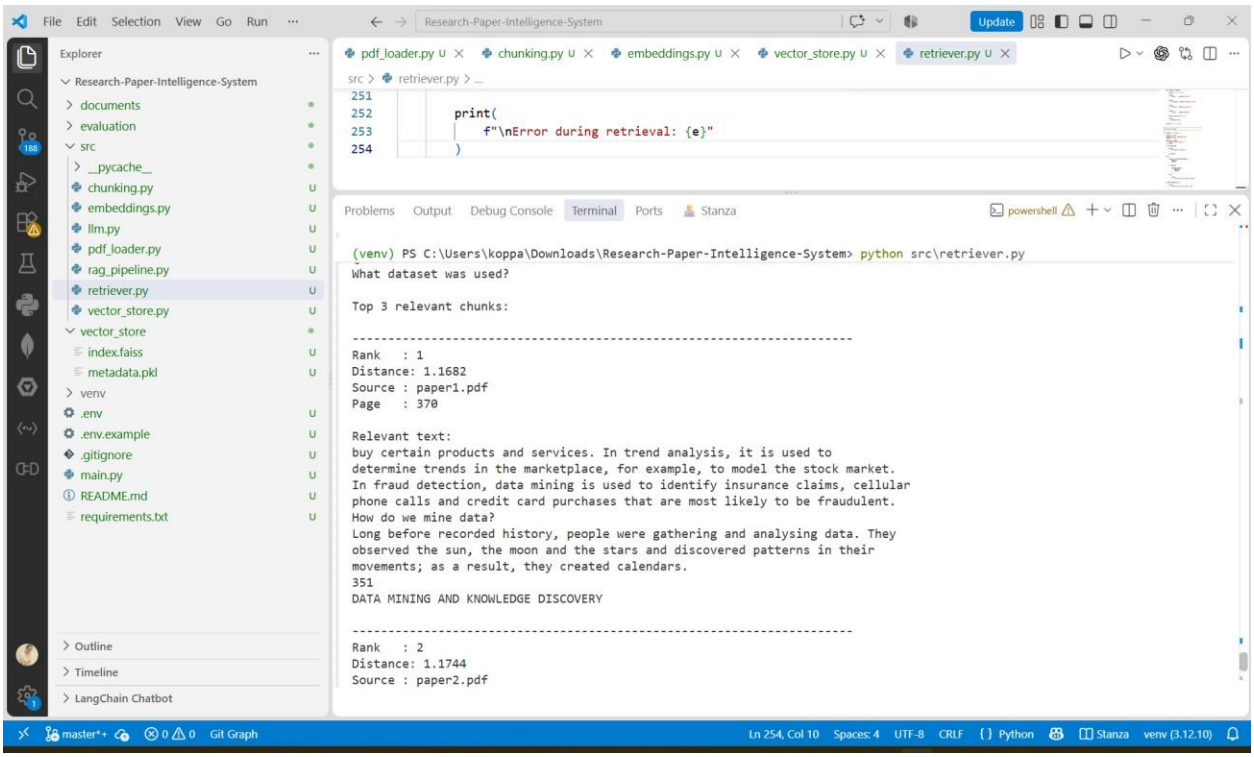
Flow:

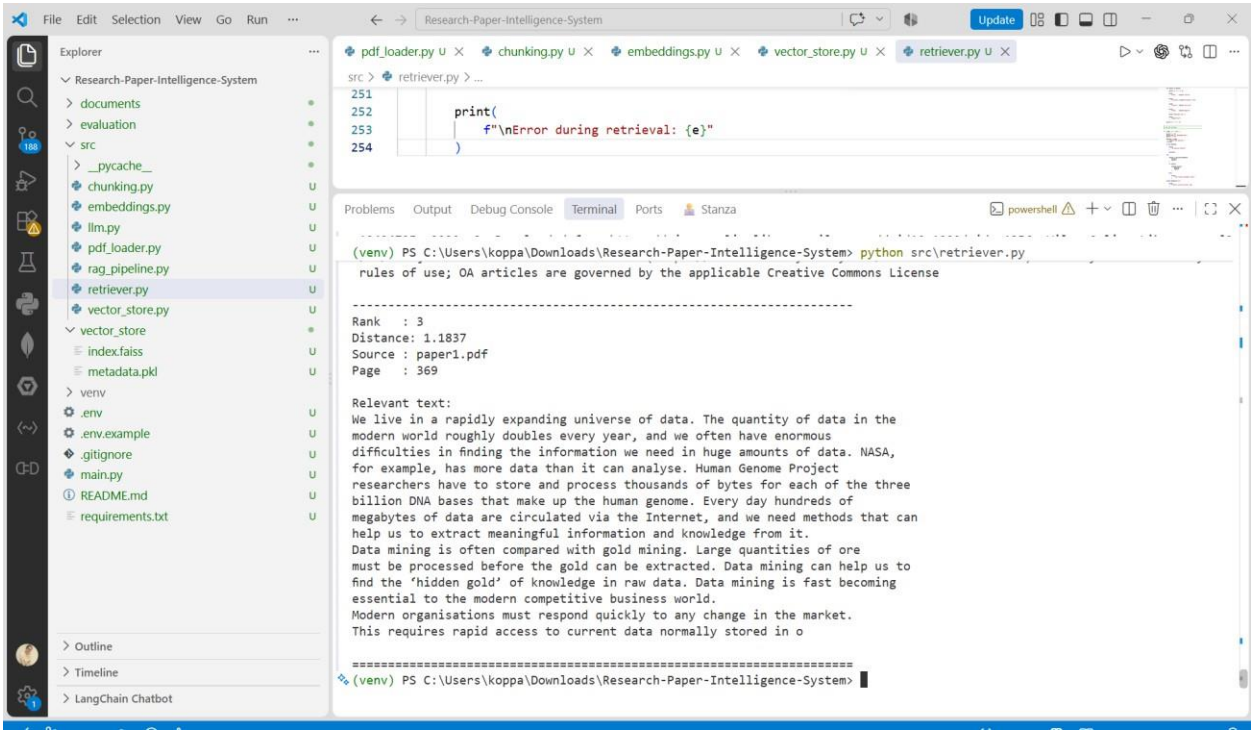
```
Question
 ↓
Question Embedding
 ↓
FAISS Search
 ↓
Top 3 Relevant Chunks
```

Return:

```
Source: paper1.pdf
Page: 6
Relevant text: ...
```




File Edit Selection View Go Run
Research-Paper-Intelligence-System
Update
Explorer
Research-Paper-Intelligence-System
documents
evaluation
src
__pycache__
chunking.py
embeddings.py
llm.py
pdf_loader.py
rag_pipeline.py
retriever.py
vector_store.py
vector_store
index.faiss
metadata.pkl
venv
.env
.env.example
.gitignore
main.py
README.md
requirements.txt
Outline
Timeline
LangChain Chatbot
pdf_loader.py U
chunking.py U
embeddings.py U
vector_store.py U
retriever.py U
src > retriever.py > ...
print(
f"\nError during retrieval: {e}"
)
Problems Output Debug Console Terminal Ports Stanza
powershell
(venv) PS C:\Users\koppa\Downloads\Research-Paper-Intelligence-System> python src\retriever.py
What dataset was used?
Top 3 relevant chunks:
Rank : 1
Distance: 1.1682
Source : paper1.pdf
Page : 370
Relevant text:
buy certain products and services. In trend analysis, it is used to
determine trends in the marketplace, for example, to model the stock market.
In fraud detection, data mining is used to identify insurance claims, cellular
phone calls and credit card purchases that are most likely to be fraudulent.
How do we mine data?
Long before recorded history, people were gathering and analysing data. They
observed the sun, the moon and the stars and discovered patterns in their
movements; as a result, they created calendars.
351
DATA MINING AND KNOWLEDGE DISCOVERY
Rank : 2
Distance: 1.1744
Source : paper2.pdf
master*+
0 0
Git Graph
Ln 254, Col 10
Spaces: 4
UTF-8
CRLF
Python
Stanza
venv (3.12.10)


File Edit Selection View Go Run
Research-Paper-Intelligence-System
Update
Explorer
Research-Paper-Intelligence-System
documents
evaluation
src
__pycache__
chunking.py
embeddings.py
llm.py
pdf_loader.py
rag_pipeline.py
retriever.py
vector_store.py
vector_store
index.faiss
metadata.pkl
venv
.env
.env.example
.gitignore
main.py
README.md
requirements.txt
Outline
Timeline
LangChain Chatbot
pdf_loader.py U
chunking.py U
embeddings.py U
vector_store.py U
retriever.py U
src > retriever.py > ...
print(
f"\nError during retrieval: {e}"
)
Problems Output Debug Console Terminal Ports Stanza
powershell
(venv) PS C:\Users\koppa\Downloads\Research-Paper-Intelligence-System> python src\retriever.py
rules of use; OA articles are governed by the applicable Creative Commons License
Rank : 3
Distance: 1.1837
Source : paper1.pdf
Page : 369
Relevant text:
We live in a rapidly expanding universe of data. The quantity of data in the
modern world roughly doubles every year, and we often have enormous
difficulties in finding the information we need in huge amounts of data. NASA,
for example, has more data than it can analyse. Human Genome Project
researchers have to store and process thousands of bytes for each of the three
billion DNA bases that make up the human genome. Every day hundreds of
megabytes of data are circulated via the Internet, and we need methods that can
help us to extract meaningful information and knowledge from it.
Data mining is often compared with gold mining. Large quantities of ore
must be processed before the gold can be extracted. Data mining can help us to
find the 'hidden gold' of knowledge in raw data. Data mining is fast becoming
essential to the modern competitive business world.
Modern organisations must respond quickly to any change in the market.
This requires rapid access to current data normally stored in o
(venv) PS C:\Users\koppa\Downloads\Research-Paper-Intelligence-System>

STEP 9 — Connect Gemini LLM

Create:

`src/llm.py`

Create:

`.env`

Add:

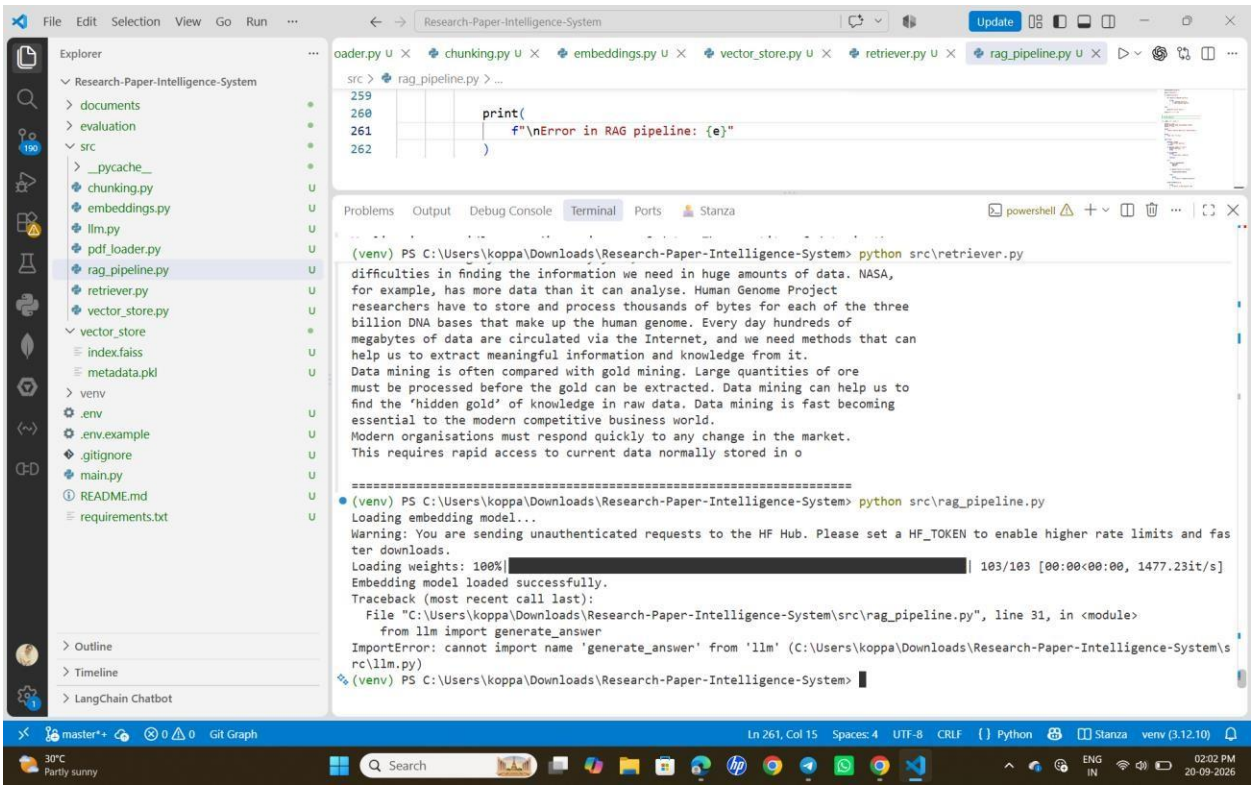
```
GEMINI_API_KEY=YOUR_API_KEY
```

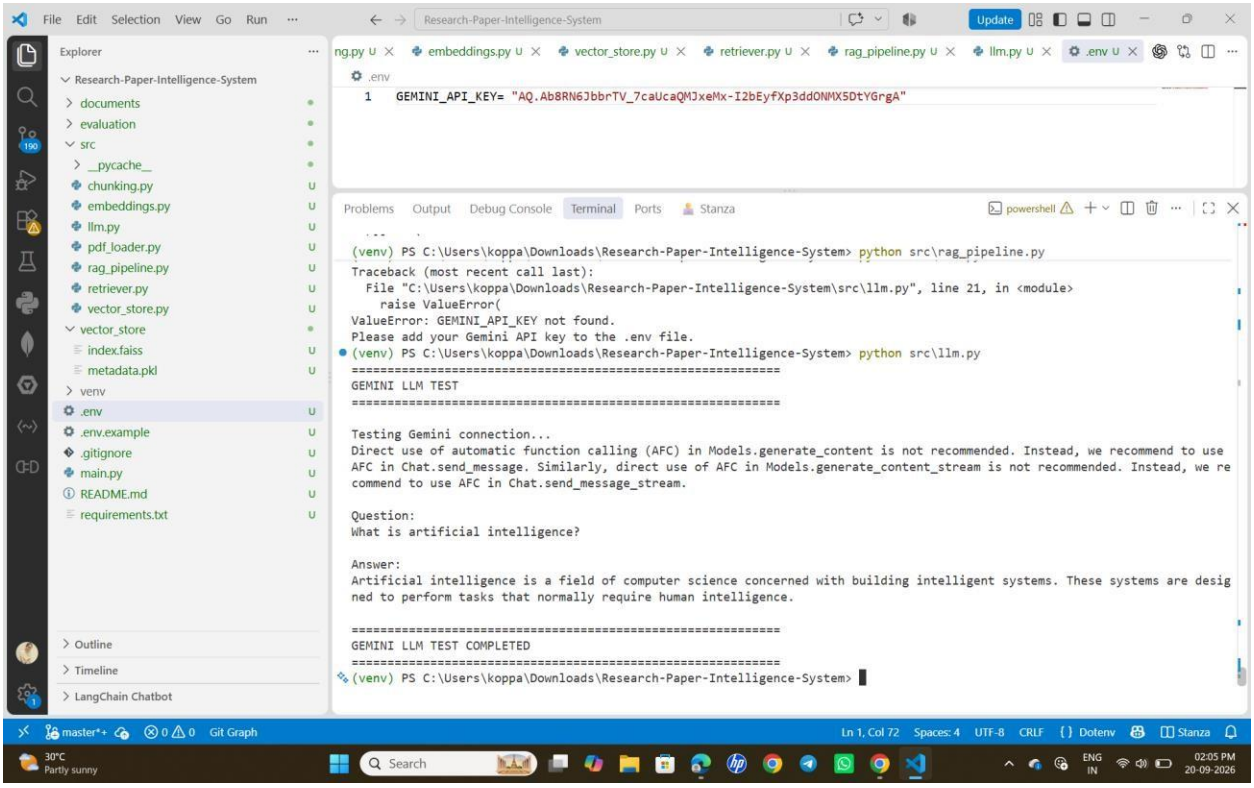
Do **not** upload `.env` to GitHub.

The LLM receives:

```
User Question
+
Retrieved Research Context
```

and generates the answer.


File Edit Selection View Go Run
Research-Paper-Intelligence-System
Update
Explorer
Research-Paper-Intelligence-System
documents
evaluation
src
__pycache__
chunking.py
embeddings.py
llm.py
pdf_loader.py
rag_pipeline.py
retriever.py
vector_store.py
vector_store
index.faiss
metadata.pkl
venv
.env
.env.example
.gitignore
main.py
README.md
requirements.txt
Outline
Timeline
LangChain Chatbot
oader.py U
chunking.py U
embeddings.py U
vector_store.py U
retriever.py U
rag_pipeline.py U
src > rag_pipeline.py > ...
259
260
print(
261
f"\nError in RAG pipeline: {e}"
262
)
Problems Output Debug Console Terminal Ports Stanza
powershell
(venv) PS C:\Users\koppa\Downloads\Research-Paper-Intelligence-System> python src\retriever.py
difficulties in finding the information we need in huge amounts of data. NASA,
for example, has more data than it can analyse. Human Genome Project
researchers have to store and process thousands of bytes for each of the three
billion DNA bases that make up the human genome. Every day hundreds of
megabytes of data are circulated via the Internet, and we need methods that can
help us to extract meaningful information and knowledge from it.
Data mining is often compared with gold mining. Large quantities of ore
must be processed before the gold can be extracted. Data mining can help us to
find the 'hidden gold' of knowledge in raw data. Data mining is fast becoming
essential to the modern competitive business world.
Modern organisations must respond quickly to any change in the market.
This requires rapid access to current data normally stored in o
(venv) PS C:\Users\koppa\Downloads\Research-Paper-Intelligence-System> python src\rag_pipeline.py
Loading embedding model...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fas
ter downloads.
Loading weights: 100%| 103/103 [00:00<00:00, 1477.23it/s]
Embedding model loaded successfully.
Traceback (most recent call last):
File "C:\Users\koppa\Downloads\Research-Paper-Intelligence-System\src\rag_pipeline.py", line 31, in <module>
from llm import generate_answer
ImportError: cannot import name 'generate_answer' from 'llm' (C:\Users\koppa\Downloads\Research-Paper-Intelligence-System\s
rc\llm.py)
(venv) PS C:\Users\koppa\Downloads\Research-Paper-Intelligence-System>
master*+ 0 0 Git Graph
Ln 261, Col 15 Spaces: 4 UTF-8 CRLF Python Stanza venv (3.12.10)
30°C Partly sunny
Search
ENG IN
02:02 PM 20-09-2026


File Edit Selection View Go Run
Research-Paper-Intelligence-System
Update
Explorer
Research-Paper-Intelligence-System
documents
evaluation
src
__pycache__
chunking.py
embeddings.py
llm.py
pdf_loader.py
rag_pipeline.py
retriever.py
vector_store.py
vector_store
index.faiss
metadata.pkl
venv
.env
.env.example
.gitignore
main.py
README.md
requirements.txt
Outline
Timeline
LangChain Chatbot
ng.py U
embeddings.py U
vector_store.py U
retriever.py U
rag_pipeline.py U
llm.py U
.env U
.env
1 GEMINI_API_KEY= "AQ.Ab8RN6JbbrTV_7caUcaQMJxeMx-I2bEyfXp3ddONMX5DtYGrgA"
Problems Output Debug Console Terminal Ports Stanza
powershell
(venv) PS C:\Users\koppa\Downloads\Research-Paper-Intelligence-System> python src\rag_pipeline.py
Traceback (most recent call last):
File "C:\Users\koppa\Downloads\Research-Paper-Intelligence-System\src\llm.py", line 21, in <module>
raise ValueError(
ValueError: GEMINI_API_KEY not found.
Please add your Gemini API key to the .env file.
(venv) PS C:\Users\koppa\Downloads\Research-Paper-Intelligence-System> python src\llm.py
==
GEMINI LLM TEST
==
Testing Gemini connection...
Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use
AFC in Chat.send_message. Similarly, direct use of AFC in Models.generate_content_stream is not recommended. Instead, we re
commend to use AFC in Chat.send_message_stream.
Question:
What is artificial intelligence?
Answer:
Artificial intelligence is a field of computer science concerned with building intelligent systems. These systems are desig
ned to perform tasks that normally require human intelligence.
==
GEMINI LLM TEST COMPLETED
==
(venv) PS C:\Users\koppa\Downloads\Research-Paper-Intelligence-System>
master*+ 0 0 Git Graph
Ln 1, Col 72 Spaces: 4 UTF-8 CRLF Dotenv Stanza
30°C Partly sunny
Search
ENG IN
02:05 PM 20-09-2026

STEP 10 — Build the complete RAG pipeline

Create:

`src/rag_pipeline.py`

Connect everything:

```
User Question
      ↓
Query Embedding
      ↓
FAISS Retrieval
      ↓
Relevant Chunks
      ↓
Context
      ↓
Gemini
      ↓
Answer
      ↓
Sources + Page Numbers
```

This is the **main RAG pipeline**.

STEP 11 — Create `main.py`

`main.py` will be your main application.

Run:

```
python main.py
```

Example:

```
========================================
Research Paper Intelligence System
========================================
```

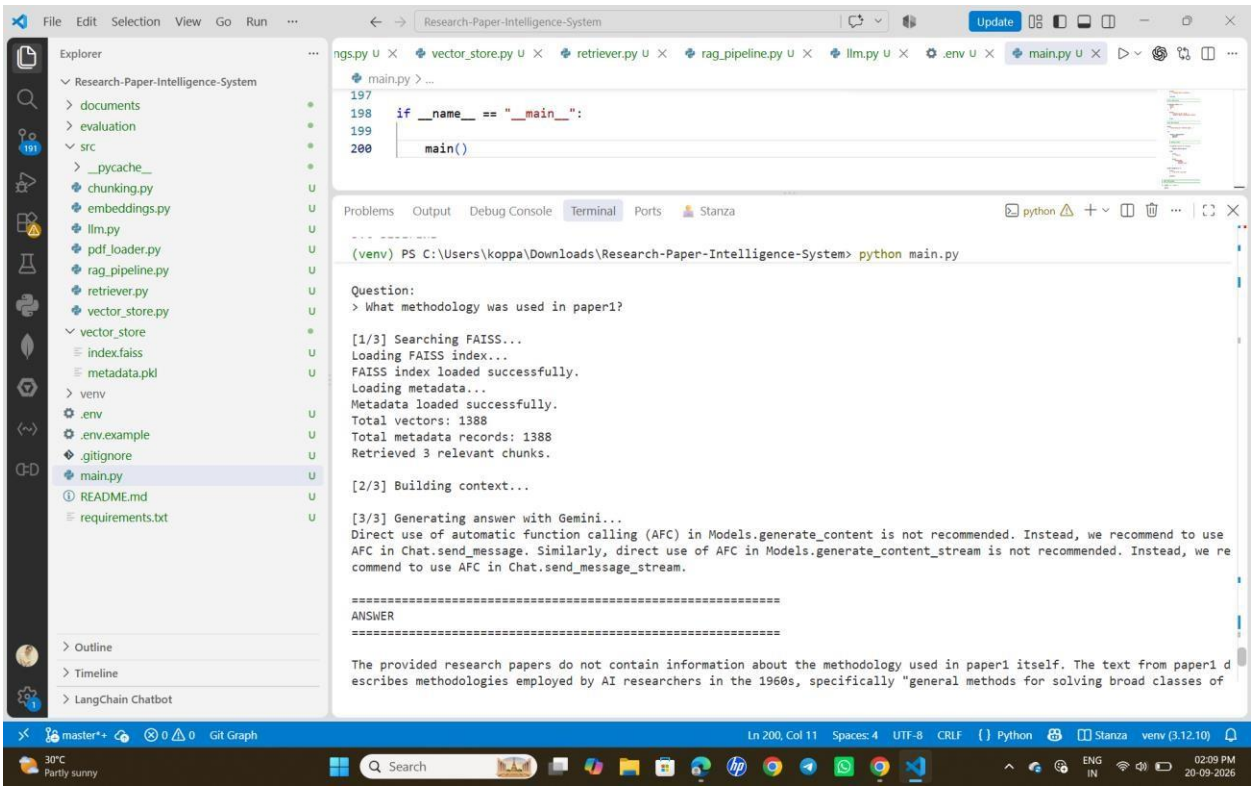
```
Enter your question:

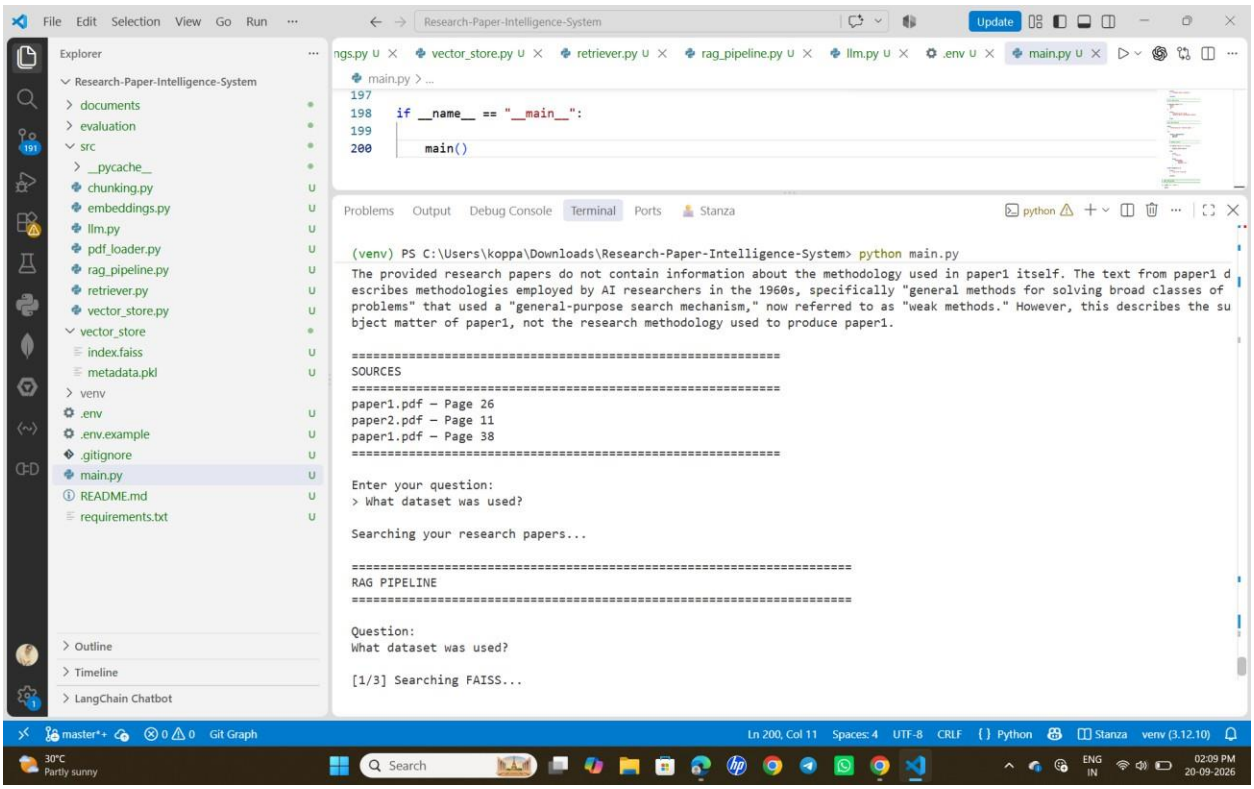
> What methodology was used in paper1?
```

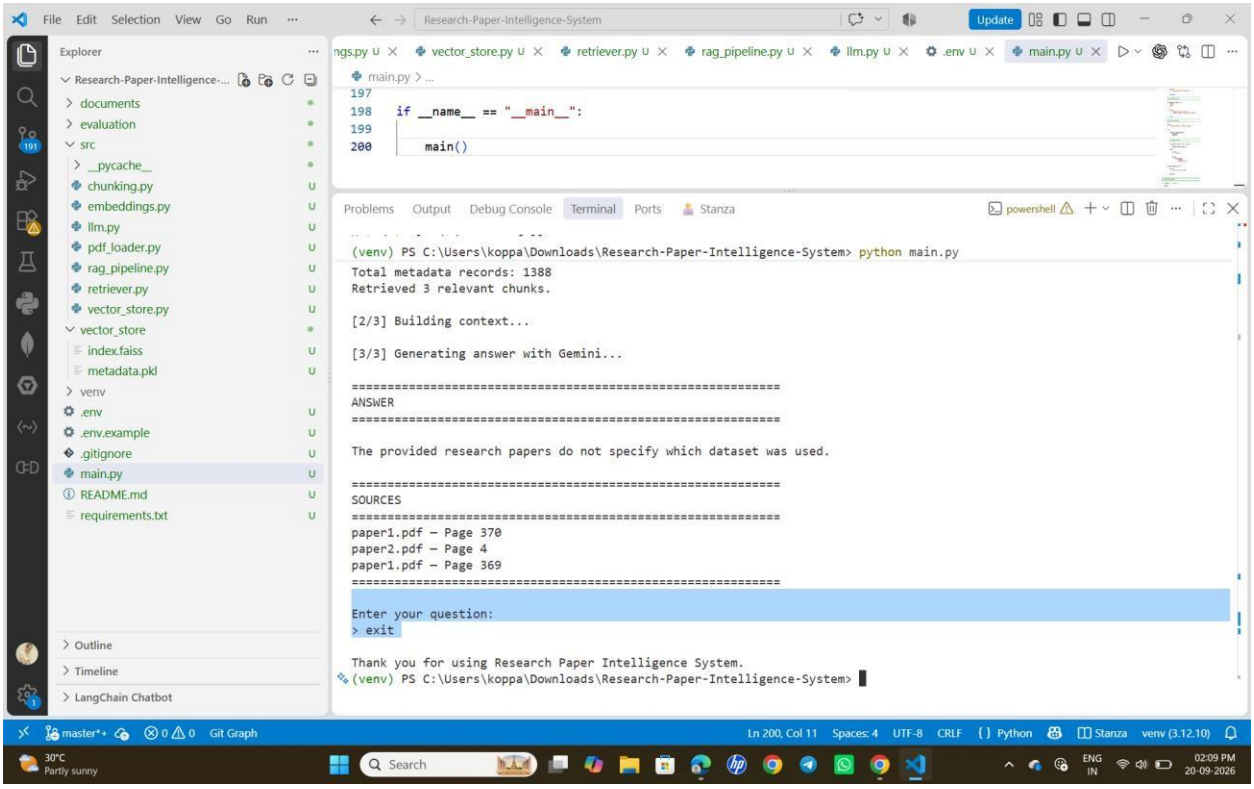
Output:

```
ANSWER
----------------------------------------------------------------
The paper uses ...

SOURCES
----------------------------------------------------------------
paper1.pdf – Page 5
paper1.pdf – Page 6
```







Enter the **exit** in the “enter your question:” then it will close the termainal

STEP 12 — Add multiple-paper questions

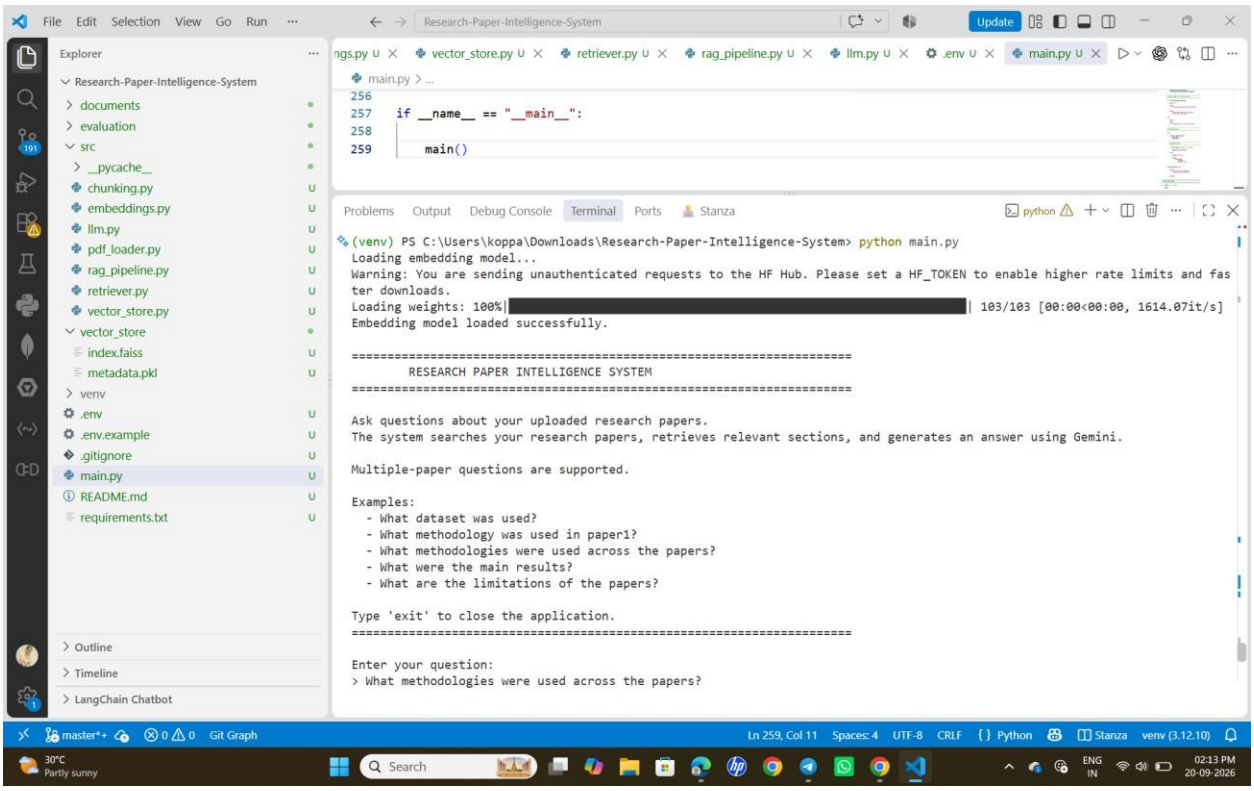
Now test:

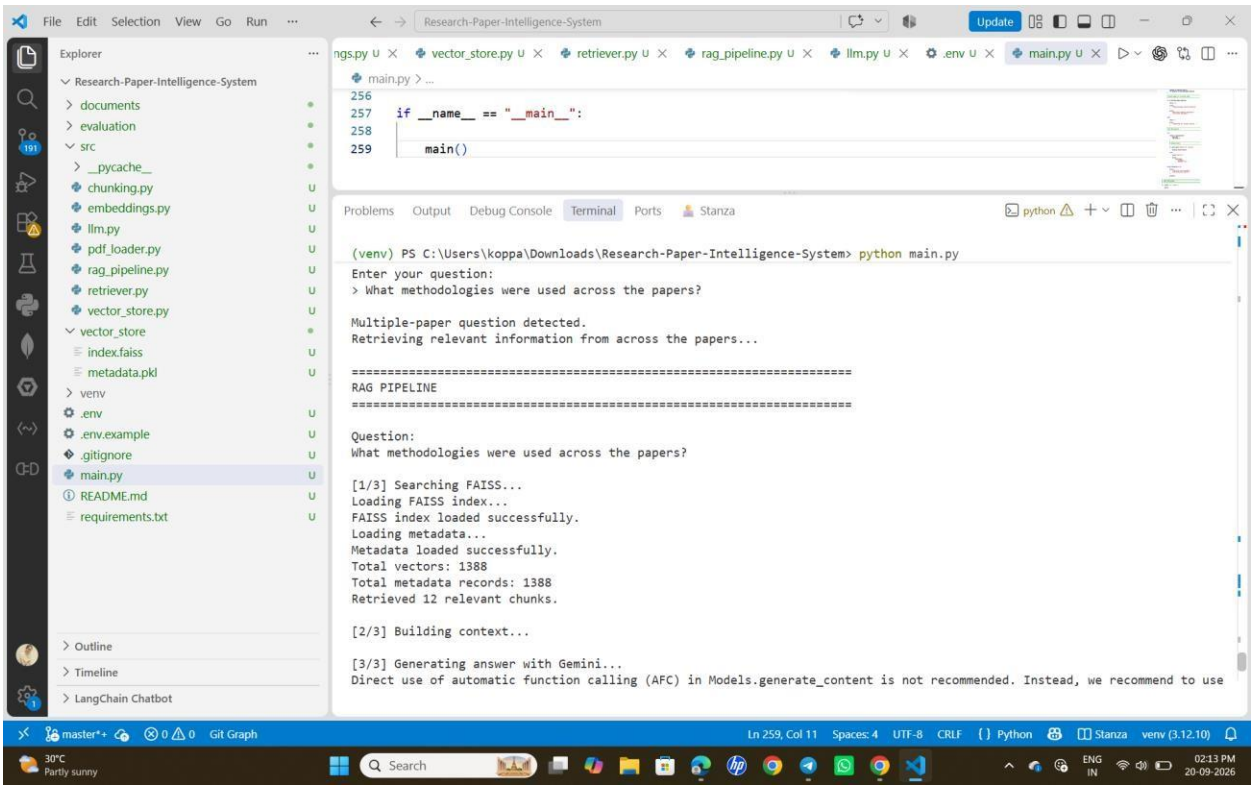
```
What methodologies were used across the papers?
```

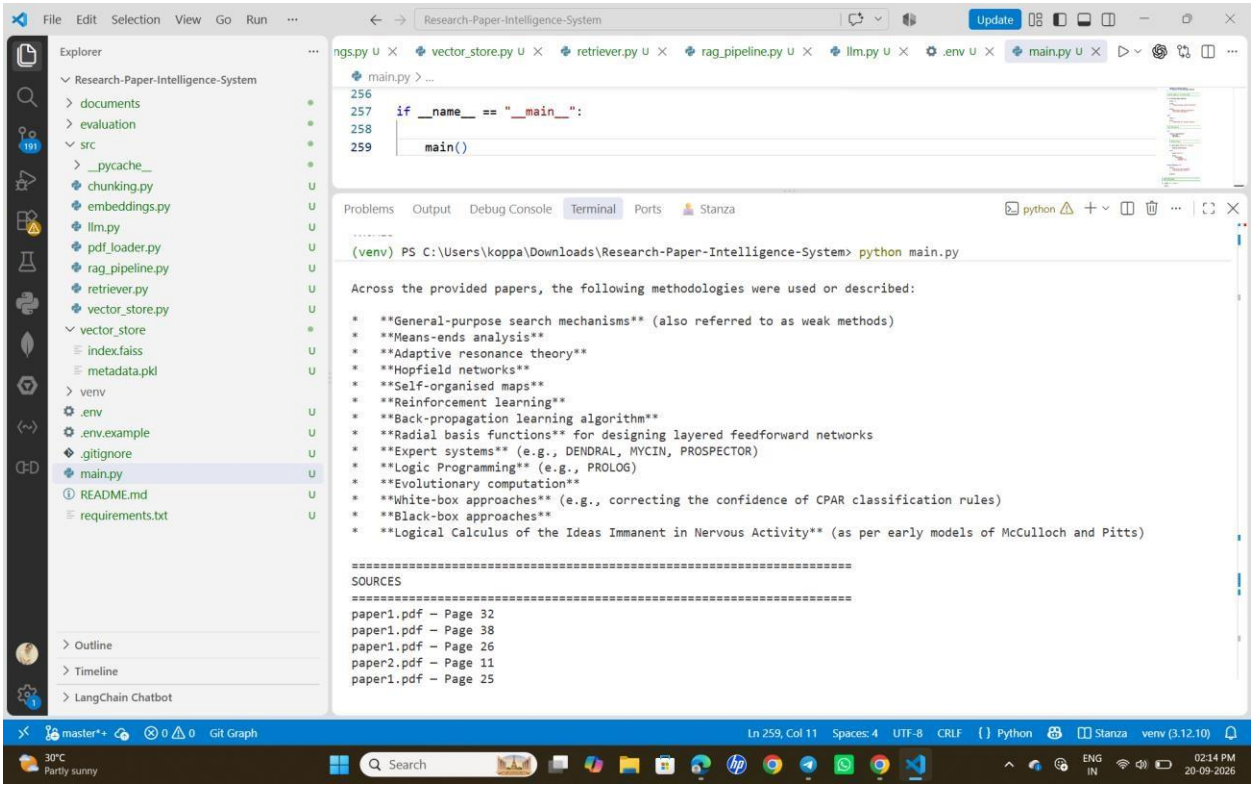
The system should search:

```
paper1.pdf
paper2.pdf
paper3.pdf
paper4.pdf
```

and provide a combined answer.




File Edit Selection View Go Run
Research-Paper-Intelligence-System
Update
Explorer
Research-Paper-Intelligence-System
documents
evaluation
src
__pycache__
chunking.py
embeddings.py
llm.py
pdf_loader.py
rag_pipeline.py
retriever.py
vector_store.py
vector_store
index.faiss
metadata.pkl
venv
.env
.env.example
.gitignore
main.py
README.md
requirements.txt
Outline
Timeline
LangChain Chatbot
vector_store.py
retriever.py
rag_pipeline.py
llm.py
.env
main.py
256
257 if __name__ == "__main__":
258
259 main()
Problems Output Debug Console Terminal Ports Stanza
python
(venv) PS C:\Users\koppa\Downloads\Research-Paper-Intelligence-System> python main.py
Enter your question:
> What methodologies were used across the papers?
Multiple-paper question detected.
Retrieving relevant information from across the papers...
RAG PIPELINE
Question:
What methodologies were used across the papers?
[1/3] Searching FAISS...
Loading FAISS index...
FAISS index loaded successfully.
Loading metadata...
Metadata loaded successfully.
Total vectors: 1388
Total metadata records: 1388
Retrieved 12 relevant chunks.
[2/3] Building context...
[3/3] Generating answer with Gemini...
Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use
master*+ 0 0 Git Graph
Ln 259, Col 11 Spaces: 4 UTF-8 CRLF Python Stanza venv (3.12.10)
30°C Partly sunny
Search
ENG IN
02:13 PM 20-09-2026


File Edit Selection View Go Run
Research-Paper-Intelligence-System
Update
Explorer
Research-Paper-Intelligence-System
documents
evaluation
src
__pycache__
chunking.py
embeddings.py
llm.py
pdf_loader.py
rag_pipeline.py
retriever.py
vector_store.py
vector_store
index.faiss
metadata.pkl
venv
.env
.env.example
.gitignore
main.py
README.md
requirements.txt
Outline
Timeline
LangChain Chatbot
vector_store.py
retriever.py
rag_pipeline.py
llm.py
.env
main.py
256
257 if __name__ == "__main__":
258
259 main()
Problems Output Debug Console Terminal Ports Stanza
python
(venv) PS C:\Users\koppa\Downloads\Research-Paper-Intelligence-System> python main.py
Across the provided papers, the following methodologies were used or described:
* **General-purpose search mechanisms** (also referred to as weak methods)
* **Means-ends analysis**
* **Adaptive resonance theory**
* **Hopfield networks**
* **Self-organised maps**
* **Reinforcement learning**
* **Back-propagation learning algorithm**
* **Radial basis functions** for designing layered feedforward networks
* **Expert systems** (e.g., DENDRAL, MYCIN, PROSPECTOR)
* **Logic Programming** (e.g., PROLOG)
* **Evolutionary computation**
* **White-box approaches** (e.g., correcting the confidence of CPAR classification rules)
* **Black-box approaches**
* **Logical Calculus of the Ideas Immanent in Nervous Activity** (as per early models of McCulloch and Pitts)
SOURCES
paper1.pdf — Page 32
paper1.pdf — Page 38
paper1.pdf — Page 26
paper2.pdf — Page 11
paper1.pdf — Page 25
master*+ 0 0 Git Graph
Ln 259, Col 11 Spaces: 4 UTF-8 CRLF Python Stanza venv (3.12.10)
30°C Partly sunny
Search
ENG IN
02:14 PM 20-09-2026

STEP 13 → Flask Backend

STEP 14 → API endpoints

STEP 15 → PDF upload API

STEP 16 → React frontend (if you want one)

STEP 17 → Paper summarization

STEP 18 → Paper comparison

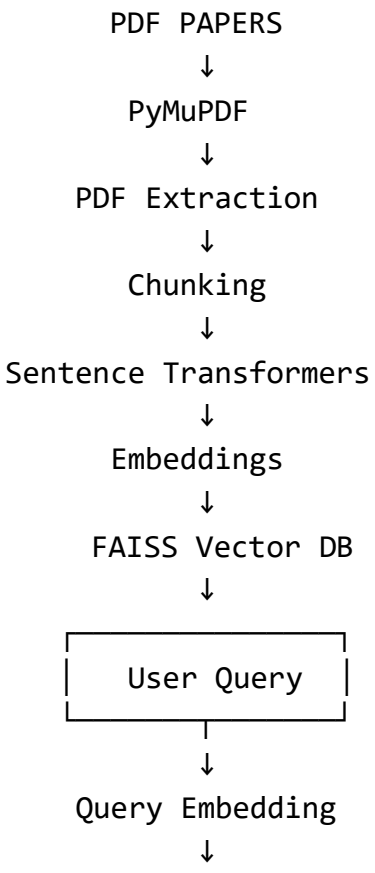
STEP 19 → Methodology / dataset / results extraction

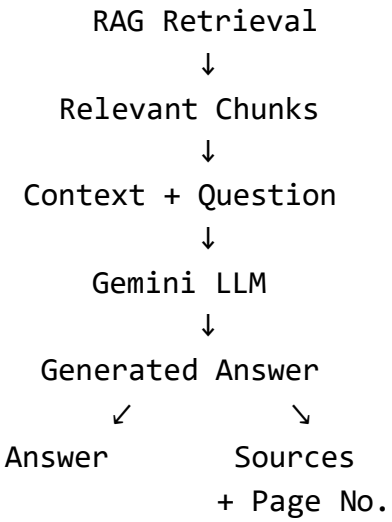
STEP 20 → Research-gap generation

STEP 21 → Evaluation

STEP 22 → README + GitHub

Final project flow





Development order to follow

1. PDF Extraction
↓
2. Chunking
↓
3. Embeddings
↓
4. FAISS
↓
5. Retrieval
↓
6. Gemini
↓
7. RAG Pipeline
↓
8. main.py
↓
9. Multi-Paper Q&A
↓
10. Summarization
↓
11. Comparison
↓
12. Information Extraction
↓

13. Research Gaps
↓
14. Evaluation

Do not start with all features simultaneously. First get **PDF → chunks → embeddings → FAISS → retrieval** working. Then add Gemini. Once that basic RAG pipeline works, add the advanced features one by one.

FINAL RESULTS:

